

算法伪代码

对应实现：`core/expert_tree.py`、`domains/geochem/experts/*.py`、`domains/geochem/sources/*.py`、`domains/geochem/prospectivity_model.py`、`domains/geochem/narrative.py`。符号： x = 查询点坐标， S = 数据源集合， E = 专家集合， P = 正样本， B = 背景样本。

算法 1 特征提取 (`sources/assay.py` , `geophysics.py` , `geology.py`)

```
FUNCTION ExtractFeatures(x, source s) -> dict[name -> value]

IF s 是化探源:                                     # 5 类：水系沉积物/岩屑/钻孔/浅钻/土壤
  FOR r IN {5, 10, 50} km:
    N_r <- KDTREE(s).query_ball(x, r)             # 邻域样品
    FOR 每个元素 e:
      v <- log1p(N_r 中 e 的浓度), 剔除非有限与负值
      IF |v| < MIN_SAMPLES: 全部记 NaN
      ELSE:
        f[e_{r}km_median] <- median(v)
        f[e_{r}km_p90] <- percentile(v, 90)
        f[e_{r}km_cv] <- std(v)/mean(v)
        f[e_{r}km_frac_above] <- mean(v >  $\theta_e$ ) #  $\theta_e$  = 全域 e 的 log 值 75 分位
      f[n_samples_{r}km] <- |N_r|
    FOR 每个元素 e:
      f[e_contrast_5_50] <- f[e_5km_median] - f[e_50km_median] # log 空间差 = 浓度比的对数
      f[e_contrast_10_50] <- f[e_10km_median] - f[e_50km_median]
    FOR 每个配置比值 (a,b):
      f[ratio_a_b] <- f[a_10km_median] - f[b_10km_median]
    f[spatial_coherence_10km] <- 主元素前 5 高值点的平均两两距离
    f[PC{i}_{5,10}km_median] <- 邻域内 PCA 主成分中位数

IF s 是地球物理栅格源:
  FOR name IN {mag, grav, K, Th, U, LuHf, SmNd}:
    f[name] <- 栅格在 x 处采样
    f[name_grad] <- sqrt(dx2 + dy2) # 水平梯度模
  f[K_Th], f[Th_U], f[U_K] <- 对应比值

IF s 是构造/地质矢量源:
  f[dist_fault_km], f[fault_density_5km], f[fault_density_10km]
  f[dist_worm_mag_km], f[dist_worm_grav_km]
  f[is_rt_*] (岩性 one-hot), f[age_ma], f[is_yilgarn], f[is_pilbara], f[is_covered]

RETURN f
```

算法 2 叶专家：拟合与打分 (`experts/geochem_experts.py` , `geophysics_experts.py`)

```

FUNCTION Expert.Fit(P, B)                                # 每个专家独立拟合
FOR 每个该专家声明的源 sn IN required_sources:
  F_P <- {ExtractFeatures(x, sn) : x ∈ P}
  F_B <- {ExtractFeatures(x, sn) : x ∈ B}
  IF F_B 为空: 跳过该源
  FOR 每个特征 f:
    μ[sn][f], σ[sn][f] <- mean, std of F_B[f] # 背景定义"正常", σ<1e-9 记 1
    w[sn][f] <- 领域先验                        # 富集专家等权; pathfinder 用先验权重字典
    d[sn][f] <- +1 IF median(F_P[f]) >= median(F_B[f]) ELSE -1 # 方向由数据学
  auc <- AUC(训练集上该源单独打分, 标签)
  IF auc < 0.5: auc <- 1-auc; d[sn][*] <- -d[sn][*] # 该源整体反向
  srcw[sn] <- max(0, (auc - 0.5) × 2) # 源权重: 判别力线性映射
END FOR

FUNCTION Expert.Score(x) -> NodeScore | None
FOR 每个 srcw[sn] > 0 的源:
  f <- ExtractFeatures(x, sn)
  IF f 不存在: 跳过
  z[sn][k] <- (f[k] - μ[sn][k]) / σ[sn][k] × d[sn][k] # 方向已并入 z
  raw[sn] <- Σ_k w[sn][k]·z[sn][k] / Σ_k w[sn][k] # 仅对有限特征求和; NaN 不进分子分母
  IF raw[sn] 无有效项: 跳过该源
  s[sn] <- sigmoid(raw[sn])
  IF 无任何可用源: RETURN None # ← 弃权: 整个专家退出聚合
score <- Σ_sn srcw[sn]·s[sn] / Σ_sn srcw[sn]
# 逐特征权重: 链式归一化, 使 Σ w·z 可重建该源的 raw
feature_w["sn:k"] <- (srcw[sn]/Σ srcw) × (w[sn][k] / Σ_{k'∈present} w[sn][k'])
RETURN NodeScore(score, feature_z=z, feature_w=feature_w, weights=srcw)

```

算法 3 专家树：拟合与打分 (core/expert_tree.py)

```

FUNCTION CompositeNode.Fit(P, B)
FOR 每个子专家 c ∈ E:
  c.Fit(P, B)
  scores <- [c.Score(x).score : x ∈ P ∪ B, 非 None]
  IF 有效样本 < 10 或单一类别: W[c] <- 0; CONTINUE
  auc <- AUC(scores, labels)
  IF auc < 0.5: auc <- 1 - auc; c.invert <- TRUE # 节点自身持有翻转, 祖先不再翻
  W[c] <- max(0, (auc - 0.5) × 2) # 全局权重

FUNCTION CompositeNode.Score(x) -> NodeScore | None
children <- {c: c.Score(x) for c ∈ E, 结果非 None} # 缺源的专家在此消失
IF children 为空: RETURN None
FOR c ∈ children:
  eff[c] <- W[c] × c.local_reliability(x) # 默认 1.0
IF Σ eff < 1e-9:
  score <- mean(children 的分数); confidence <- 0
ELSE:
  score <- Σ_c eff[c]·children[c].score / Σ_c eff[c]
  confidence <- Σ_c eff[c] / Σ_{c ∈ E} W[c] # 实际动用的权重占比

```

```
IF self.invert: score <- 1 - score
RETURN NodeScore(score, confidence, children, weights=eff)
```

算法 4 端到端打分与叙述 (prospectivity_model.py , narrative.py)

```
FUNCTION Fit(catalog, config, exclude_ids)
  active <- 覆盖率探测通过的源 # 40 个随机点, 10 km 内 ≥3 样品, 通过率 ≥10%
  P <- 已知矿点(config.sites_csv) 去掉 exclude_ids
  B <- 均匀随机撒点(WA bbox) 且 10 km 内 ≥3 样品, 取 n_bg=300 个, seed 固定
  E <- 3 个化探专家(跨全部化探源) + 4 个地物专家 + 3 个构造专家
  tree <- CompositeNode(E); tree.Fit(P, B)

FUNCTION Explain(x)
  ns <- tree.Score(x)
  tier <- Strong IF ns.score ≥ 0.80 ELSE Moderate IF ≥ 0.65 ELSE Weak IF ≥ 0.52 ELSE Background
  FOR 每个子专家 c:
    contrib[c] <- eff[c]/Σ eff × |ns.children[c].score - 0.5| # 专家贡献排序
  FOR 每个特征 "sn:k":
    attribution <- (eff[c]/Σ eff) × feature_w["sn:k"] × feature_z["sn:k"]
  RETURN PointNarrative(score, tier, confidence, ranked contrib, ranked attribution)
  # 结构化记录, 非字符串; LLM 只能复述其中的数字
```

恒等式 (已实测, 9/9 金属误差为 0) : $ns.score = \sum_c (eff[c]/\Sigma eff) \cdot \text{sigmoid}(\sum_k feature_w[c][k] \cdot feature_z[c][k])$

算法 5 评测协议 (scripts/eval_comprehensive.py , eval_baselines.py)

```
FOR 每个金属:
  all_pos <- SITE_CODE 中含 "Mine|Deposit" 的记录
  test_pos <- 固定置换(seed=42) 的前 30 个; train_pos <- 其余
  forbid <- test_pos 的坐标 ( km )

  # 训练集: 所有模型共用
  train_neg <- 随机 WA 点, 有化探覆盖, 距 test_pos ≥ 5 km, n=300, seed=142

  # 四种测试负样本
  random <- 随机 WA 点, 有覆盖, 距 test_pos ≥5 km (n=200, seed=43)
  far_random <- 同上 且 距任何已知矿点 ≥50 km (n=200, seed=44)
  true_nonmine <- no_<el>_sites.csv: 他矿种的真实矿点、本矿种缺失 (n≤200, seed=45)
  spatial <- 南(Y<-25)训练 / 北(Y≥-25)测试, 负样本取北部非矿点 (n=200, seed=47)

  # 模型
  expert_tree <- Fit(train_pos, train_neg) # 算法 4
  X <- 全部活跃源的特征表, 列名 "source:feature"
  X <- 仅保留 10 个专家实际读取的 195 列 ( expert_matched ), 训练集中位数填补 NaN
  FOR m IN {DecisionTree(depth=4), RandomForest(500), XGBoost(400,depth=4), L1-Logistic}:
    m.fit(X[train], y[train])

  FOR 每个测试场景: 报告 AUC 与 P@{10,20,50}
```

算法 6 解释质量评测 (scripts/eval_explanation_quality.py)

```
# 归因：各模型用各自的，不做交叉强加
A[expert_tree] <- eff[c]/Σeff × |feature_w × feature_z|
A[decision_tree] <- 决策路径上各节点的加权不纯度下降
A[random_forest] <- TreeSHAP(shap.TreeExplainer)
A[xgboost] <- booster.predict(pred_contribs=True) # 精确 TreeSHAP
A[logit_l1] <- |coef × (x - 训练均值)|

# 忠实度：声称的头号驱动 vs 实际影响最大者
FOR 每个特征 j: # 195 个全测，不抽样
  E[:,j] <- |score(x) - score(x 中第 j 维替换为训练中位数)|
  # 专家树穿过真实模型：改数据源缓存 → 重新 score_batch → 还原
Hit@1 <- mean( argmax_j A[i,j] == argmax_j E[i,j] )
ρ <- mean_i spearman(A[i,:], E[i,:])
# 专家组级：按 10 个专家把列分组，整组掩蔽，同样计算

# 稳定性
FOR b IN 1..10:
  有放回重抽训练集 → 重新拟合 → 重新计算 A_b
  J_b <- mean_i jaccard(top3(A[i]), top3(A_b[i]))
stability <- mean_b J_b
```

算法 7 缺失模态与自报可靠性 (eval_degradation.py , eval_confidence.py)

```
FOR k IN 0..|S|-1:
  FOR rep IN 1..N_REP:
    D <- 随机抽 k 个源
    # 专家树：把 D 中各源对测试点的特征置 NaN → 相关专家 Score 返回 None → 自动弃权
    score_expert, confidence <- tree.Score(测试点 with D 置空)
    # 定长模型：缺列只能填补 ( XGBoost 另跑一版原生 NaN )
    score_ml <- m.predict(X with D 列 <- 训练中位数)
    记录 AUC(k)、mean confidence(k)、活跃专家数(k)

# 选择性预测：各模型用各自的可靠性信号
signal[expert_tree] <- confidence # 独立于分数
signal[其余] <- |p - 0.5| # 由分数导出；RF 另有树间投票标准差
FOR cov IN {100%, 75%, 50%, 25%}:
  保留 signal 最高的 cov 比例的点，计算 AUC
ρ_err <- spearman(signal, -|y - p|)
```